

南开大学

汇编语言与逆向技术课程实验报告

实验六：读取 PE 文件的导入表和导出表



学院：密码与网络空间安全学院

专业：信息安全

学号：2412266

姓名：杨滢

班级：信息安全

1 实验时间

第 13 周星期一 9-10 节

2 实验目的

熟悉 PE 文件的导入表和导出表结构；

3 实验环境

Windows 操作系统，MASM32 编译环境。

4 PE 文件导入表的作用和数据结构

4.1 作用

用于记录程序运行时所需的外部函数或变量的引用信息，通常指向动态链接库（DLL）中的函数。

4.2 数据结构

导入表位于数据目录的第二项（索引 1），指向一个 IMAGE_IMPORT_DESCRIPTOR 数组。每个导入的 DLL 对应一个描述符，内容包括：

- OriginalFirstThunk：指向导入名称表（INT），存储函数名信息；
- FirstThunk：指向导入地址表（IAT），存储函数地址；
- Name：指向 DLL 名称字符串。

INT 和 IAT 都是 IMAGE_THUNK_DATA 数组，每个元素对应一个导入函数，以 NULL 结尾。IMAGE_THUNK_DATA 结构可表示函数序号或指向 IMAGE_IMPORT_BY_NAME 结构。

```
1 typedef struct _IMAGE_IMPORT_DESCRIPTOR {
2     union {
3         DWORD Characteristics;
4         DWORD OriginalFirstThunk;
5     } DUMMYUNIONNAME;
6     DWORD TimeDateStamp;
7     DWORD ForwarderChain;
8     DWORD Name;
9     DWORD FirstThunk;
10 } IMAGE_IMPORT_DESCRIPTOR;
11 typedef IMAGE_IMPORT_DESCRIPTOR UNALIGNED *PIMAGE_IMPORT_DESCRIPTOR;
```

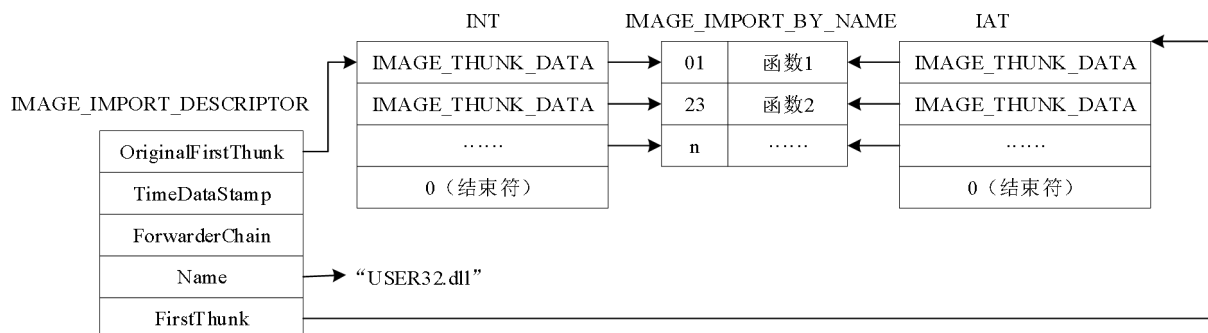


图 4.1: 导入表结构

5 PE 文件导出表的作用和数据结构

5.1 作用

用于记录当前模块导出的函数或数据，供其他模块调用。

5.2 数据结构

导出表位于数据目录的第一项（索引 0），指向一个 `IMAGE_EXPORT_DIRECTORY` 结构，内容包括：

- `NumberOfFunctions`：导出函数数量；
- `AddressOfFunctions`：导出函数地址数组；
- `AddressOfNames`：导出函数名称数组；
- `AddressOfNameOrdinals`：导出函数序号数组。

导出函数可通过名称或序号进行访问。

```

1 typedef struct _IMAGE_EXPORT_DIRECTORY {
2     DWORD Characteristics;
3     DWORD TimeDateStamp;
4     WORD MajorVersion;
5     WORD MinorVersion;
6     DWORD Name;
7     DWORD Base;
8     DWORD NumberOfFunctions;
9     DWORD NumberOfNames;
10    DWORD AddressOfFunctions;
11    DWORD AddressOfNames;
12    DWORD AddressOfNameOrdinals;
13 } IMAGE_EXPORT_DIRECTORY, *PIMAGE_EXPORT_DIRECTORY;

```

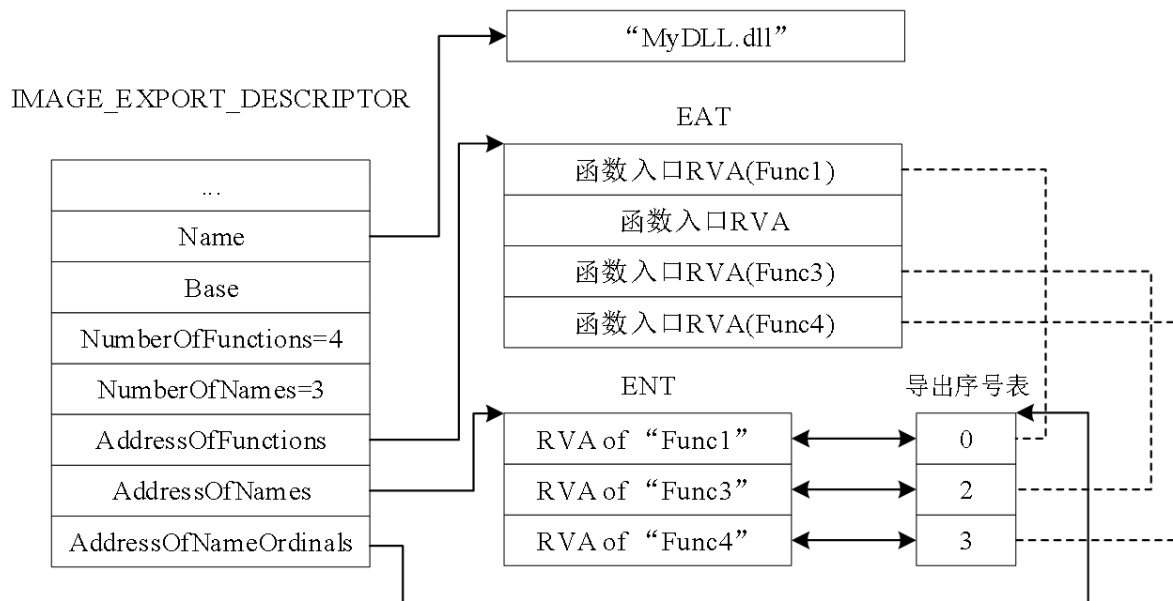


图 5.2: 导出表结构

6 源代码及注释

主要步骤:

1. 输入 PE 文件的文件名，调用 Windows API 函数，打开指定的 PE 文件
2. 读取 PE 文件的输入表，显示输入表中引入的 DLL 文件名和对应的库函数名字
3. 读入 PE 文件的输出表，显示导出函数的函数名

```

1  .386
2  .model flat, stdcall
3  option casemap :none
4
5  include \masm32\include\windows.inc
6  include \masm32\include\kernel32.inc
7  include \masm32\include\masm32.inc
8  includelib \masm32\lib\kernel32.lib
9  includelib \masm32\lib\masm32.lib
10
11 .data
12     str_Input_Cmd BYTE "Please input a PE file: ",0
13     str_Import BYTE "Import table:",0ah,0dh,0
14     str_Export BYTE "Export table:",0ah,0dh,0
15
16     inputBuf BYTE 256 DUP(0)
17     outputBuf BYTE 256 DUP(0)

```

```
18     fileBuf BYTE 4000 DUP(0)
19
20     n BYTE 0ah,0dh,0    ; 换行符
21     tab BYTE 9h,0       ; 制表符
22
23 .code
24 ; 函数: rva_to_raw, 功能为将相对虚拟地址(RVA)转换为文件偏移地址(RAW)
25 rva_to_raw PROC
26     push ebp
27     MOV ebp,esp
28     sub esp,8
29     ; 计算节表的起始地址
30     MOV eax,[ebp+8]      ; 获取NT头地址
31     movzx ecx,word ptr [eax+14h] ; 读取SizeOfOptionalHeader
32     add eax,18h          ; 跳过Signature和FileHeader
33     add eax,ecx          ; 加上可选头大小, 现在指向节表
34     MOV [ebp-4],eax      ; 保存节表起始地址到局部变量1
35     ; 获取节的数量
36     MOV eax,[ebp+8]      ; 重新获取NT头地址
37     movzx ecx,word ptr [eax+6h] ; 读取NumberOfSections
38     MOV [ebp-8],ecx      ; 保存节数量到局部变量2
39
40     MOV ebx,[ebp-4]      ; 设置遍历基地址为节表起始
41     MOV edx,[ebp+12]     ; 获取要转换的RVA值
42
43 find_loop:
44     ; 检查当前节的VirtualAddress
45     MOV eax,[ebx+0ch]    ; 获取当前节的VirtualAddress
46     cmp edx,eax          ; 比较RVA是否小于VirtualAddress
47     jb not_in_bound     ; 如果小于, 不在这个节内
48     ; 计算当前节的结束RVA
49     add eax,[ebx+10h]    ; 加上VirtualSize得到节结束RVA
50     cmp edx,eax          ; 比较RVA是否大于等于结束RVA
51     jae not_in_bound     ; 如果大于等于, 不在这个节内
52     ; RVA在当前节内, 进行转换
53     MOV eax,edx          ; 将RVA复制到eax
54     sub eax,[ebx+0ch]    ; 减去VirtualAddress得到节内偏移
55     add eax,[ebx+14h]    ; 加上PointerToRawData得到文件偏移
56     jmp find_end        ; 转换完成, 跳转到函数结束
57
58 not_in_bound:
59     ; 检查下一个节
```

```
60     add ebx,28h           ; 移动到下一个节表项(IMAGE_SECTION_HEADER大小为28h字节)
61     loop find_loop       ; 继续循环检查
62     ; 如果没有找到对应的节, 返回0
63     xor eax,eax          ; 将eax清零
64
65 find_end:
66     MOV esp,ebp
67     pop ebp
68     ret
69 rva_to_raw ENDP
70 ; 函数: printImport, 功能为解析并显示导入表信息
71 printImport PROC
72     push ebp
73     MOV ebp,esp
74     sub esp,12
75     ; 检查导入表RVA是否为0
76     MOV eax,[ebp+12]
77     cmp eax,0
78     je import_end        ; 如果为0, 没有导入表, 直接返回
79     ; 显示导入表标题
80     invoke StdOut, addr str_Import
81     ; 将导入表RVA转换为文件偏移
82     push [ebp+12]         ; 参数2: 导入表RVA
83     push [ebp+8]          ; 参数1: NT头地址
84     call rva_to_raw
85     add esp,8             ; 平衡堆栈
86     add eax,offset fileBuf ; 加上文件缓冲区基址得到实际地址
87     MOV [ebp-4],eax        ; 保存导入表文件地址到局部变量1
88     MOV [ebp-8],eax        ; 保存遍历指针到局部变量2
89
90 loop_IID:
91     ; 检查当前IID是否为空(全0表示结束)
92     MOV eax,[ebp-8]        ; 获取当前IID地址
93     MOV ecx,[eax]          ; 读取OriginalFirstThunk
94     cmp ecx,0              ; 检查是否为0
95     je loop_IID_end        ; 如果为0, 表示导入表结束
96     ; 输出一个制表符用于缩进
97     invoke StdOut,addr tab
98     ; 获取并输出DLL名称
99     MOV eax,[ebp-8]        ; 获取当前IID地址
100    MOV ebx,[eax+0ch]       ; 读取Name字段的RVA
101    push ebx                ; 参数2: Name的RVA
```

```
102     push [ebp+8]          ; 参数1: NT头地址
103     call rva_to_raw       ; 转换为文件偏移
104     add esp,8             ; 平衡堆栈
105     add eax,offset fileBuf ; 得到DLL名称在文件中的地址
106     invoke StdOut, eax    ; 输出DLL名称
107     invoke StdOut, addr n ; 换行
108     ; 获取INT(导入名称表)地址
109     MOV eax,[ebp-8]       ; 获取当前IID地址
110     MOV eax,[eax]         ; 读取OriginalFirstThunk的RVA
111     push eax              ; 参数2: OriginalFirstThunk的RVA
112     push [ebp+8]          ; 参数1: NT头地址
113     call rva_to_raw       ; 转换为文件偏移
114     add esp,8             ; 平衡堆栈
115     add eax,offset fileBuf ; 得到INT在文件中的地址
116     MOV [ebp-12],eax      ; 保存INT起始地址到局部变量3
117
118 loop_int:
119     ; 检查INT项是否为0(表示结束)
120     MOV eax,[ebp-12]      ; 获取当前INT项地址
121     MOV eax,[eax]         ; 读取INT项值(指向函数名的RVA)
122     cmp eax,0             ; 检查是否为0
123     je loop_int_end      ; 如果为0, 当前DLL的导入函数列表结束
124     ; 输出两个制表符用于函数名缩进
125     invoke StdOut,addr tab
126     invoke StdOut,addr tab
127     ; 获取并输出导入函数名称
128     MOV eax,[ebp-12]      ; 获取当前INT项地址
129     MOV eax,[eax]         ; 读取指向函数名的RVA
130     push eax              ; 参数2: 函数名RVA
131     push [ebp+8]          ; 参数1: NT头地址
132     call rva_to_raw       ; 转换为文件偏移
133     add esp,8             ; 平衡堆栈
134     add eax,offset fileBuf ; 得到IMAGE_IMPORT_BY_NAME在文件中的地址
135     add eax,2             ; 跳过Hint字段(2字节)
136     invoke StdOut, eax    ; 输出函数名称
137     invoke StdOut, addr n ; 换行
138     ; 移动到下一个INT项
139     MOV ecx,4             ; 每个INT项大小为4字节
140     add [ebp-12],ecx      ; 增加地址
141     jmp loop_int          ; 继续处理下一个函数
142
143 loop_int_end:
```

```
144     ; 移动到下一个IID
145     MOV ecx,14h           ; 每个IID大小为14h字节(20字节)
146     add [ebp-8],ecx       ; 增加地址
147     jmp loop_IID         ; 继续处理下一个DLL
148
149 loop_IID_end:
150 import_end:
151     MOV esp,ebp
152     pop ebp
153     ret
154 printImport ENDP
155
156 ; 函数: printExport, 功能为解析并显示导出表信息
157 printExport PROC
158     push ebp
159     MOV ebp,esp
160     sub esp,8
161     ; 总是显示导出表标题(即使没有导出表)
162     invoke StdOut, addr str_Export
163     ; 检查导出表RVA是否为0
164     MOV eax,[ebp+12]
165     cmp eax,0
166     je export_end         ; 如果为0, 没有导出表, 直接返回
167     ; 将导出表RVA转换为文件偏移
168     push [ebp+12]         ; 参数2: 导出表RVA
169     push [ebp+8]          ; 参数1: NT头地址
170     call rva_to_raw
171     add esp,8             ; 平衡堆栈
172     add eax,offset fileBuf ; 得到导出目录在文件中的地址
173     MOV ebx,eax           ; 保存导出目录地址到ebx
174     ; 获取导出函数数量
175     MOV eax,[eax+24]       ; 读取NumberOfNames(按名称导出的函数数量)
176     MOV [ebp-4],eax        ; 保存到局部变量1
177     ; 获取AddressOfNames数组的RVA
178     MOV eax,[ebx+32]       ; 读取AddressOfNames的RVA
179     ; 检查AddressOfNames是否为0
180     cmp eax,0
181     je export_end         ; 如果为0, 没有按名称导出的函数, 直接返回
182     ; 将AddressOfNames的RVA转换为文件偏移
183     push eax               ; 参数2: AddressOfNames的RVA
184     push [ebp+8]           ; 参数1: NT头地址
185     call rva_to_raw
```



```
186     add esp,8           ; 平衡堆栈
187     add eax,offset fileBuf ; 得到函数名地址数组在文件中的地址
188     MOV [ebp-8],eax      ; 保存到局部变量2
189     ; 设置循环计数器
190     MOV ecx,[ebp-4]      ; 获取要遍历的函数数量
191
192 export_loop:
193     ; 保存循环计数器
194     MOV [ebp-4],ecx
195     ; 输出一个制表符用于缩进
196     invoke StdOut, addr tab
197     ; 获取当前函数名的RVA
198     MOV ebx,[ebp-8]      ; 获取函数名地址数组当前位置
199     push [ebx]           ; 参数2: 函数名的RVA
200     push [ebp+8]         ; 参数1: NT头地址
201     call rva_to_raw      ; 转换为文件偏移
202     add esp,8           ; 平衡堆栈
203     add eax,offset fileBuf ; 得到函数名字符串在文件中的地址
204     invoke StdOut, eax    ; 输出函数名称
205     invoke StdOut, addr n ; 换行
206     ; 移动到下一个函数名地址
207     MOV edx,4            ; 每个地址占4字节
208     add [ebp-8],edx      ; 增加地址
209     ; 恢复循环计数器并继续
210     MOV ecx,[ebp-4]
211     loop export_loop     ; 继续处理下一个函数
212
213 export_end:
214     MOV esp,ebp
215     pop ebp
216     ret
217 printExport ENDP
218
219
220 ; 主函数
221 main PROC
222     push ebp
223     MOV ebp,esp
224     sub esp,12
225     ; 提示用户输入文件名
226     invoke StdOut, addr str_Input_Cmd
227     ; 读取用户输入的文件名
```

```
228     invoke StdIn, addr inputBuf, 255
229     ; 打开指定的文件
230     invoke CreateFile, addr inputBuf,
231             GENERIC_READ,          ; 只读访问
232             FILE_SHARE_READ,      ; 共享读取
233             0,                    ; 安全属性
234             OPEN_EXISTING,        ; 打开已存在的文件
235             FILE_ATTRIBUTE_ARCHIVE, ; 文件属性
236             0                     ; 模板文件句柄
237     MOV [ebp-4],eax                ; 保存文件句柄到局部变量1
238
239     ; 将文件指针移动到文件开头
240     invoke SetFilePointer, [ebp-4], 0, 0, FILE_BEGIN
241     ; 读取文件内容到缓冲区
242     invoke ReadFile, [ebp-4], addr fileBuf, 4000, 0, 0
243     ; 计算NT头的地址
244     ; fileBuf是DOS头地址, e_lfanew是PE头偏移
245     MOV eax,offset fileBuf ; 获取文件缓冲区起始地址
246     add eax,[eax+3ch]      ; 加上e_lfanew得到NT头偏移
247     MOV [ebp-8],eax       ; 保存NT头地址到局部变量2
248     ; 计算DataDirectory的地址
249     ; NT头偏移 + 78h是DataDirectory数组的起始
250     add eax,78h
251     MOV [ebp-12],eax      ; 保存DataDirectory地址到局部变量3
252     ; 处理导入表
253     MOV ebx,[eax+8]       ; 获取导入表RVA(DataDirectory[1])
254     push ebx              ; 参数2: 导入表RVA
255     push [ebp-8]          ; 参数1: NT头地址
256     call printImport      ; 调用导入表显示函数
257     add esp,8             ; 平衡堆栈
258     ; 在导入表和导出表之间添加一个空行
259     invoke StdOut, addr n
260     ; 处理导出表
261     MOV eax,[ebp-12]      ; 重新获取DataDirectory地址
262     MOV ebx,[eax]         ; 获取导出表RVA(DataDirectory[0])
263     push ebx              ; 参数2: 导出表RVA
264     push [ebp-8]          ; 参数1: NT头地址
265     call printExport      ; 调用导出表显示函数
266     add esp,8             ; 平衡堆栈
267     invoke CloseHandle, [ebp-4] ; 关闭文件句柄
268     invoke ExitProcess, 0    ; 退出程序
269 main ENDP
```

```
270  
271 END main
```

7 运行示例截图

```
E:\AssemblyLanguageReport>E:\AssemblyLanguageReport\import_export.exe  
Please input a PE file: import_export.exe  
Import table:  
    kernel32.dll  
        CloseHandle  
        CreateFileA  
        ExitProcess  
        ReadFile  
        SetFilePointer  
        GetStdHandle  
        WriteFile  
        SetConsoleMode  
  
Export table:  
  
E:\AssemblyLanguageReport>E:\AssemblyLanguageReport\import_export.exe  
Please input a PE file: bubble_sort.exe  
Import table:  
    kernel32.dll  
        GetStdHandle  
        WriteFile  
        ReadFile  
        SetConsoleMode  
        ExitProcess  
  
Export table:  
  
E:\AssemblyLanguageReport>E:\AssemblyLanguageReport\import_export.exe  
Please input a PE file: dec2hex.exe  
Import table:  
    kernel32.dll  
        lstrlenA  
        GetStdHandle  
        WriteFile  
        ReadFile  
        SetConsoleMode  
        ExitProcess  
  
Export table:
```

图 7.3: 运行结果

这里测试了 import_export.exe、bubble_sort.exe 和 dec2hex.exe 三个 PE 文件。

8 导入表的安全问题讨论

8.1 导入表的核心安全风险

PE 文件的导入表记录了程序运行时需从外部 DLL 加载的函数名称及地址，汇编层面直接操作导入表的指针、名称等结构，易引发以下安全问题：

1. 导入表指针篡改：导入表的 IMAGE_IMPORT_DESCRIPTOR 结构体包含指向 DLL 名称字符串的 Name 指针、指向导入函数地址表（IAT）的 FirstThunk 指针。攻击者通过修改这些指针，可将程序导向恶意 DLL 或伪造的函数地址。
2. IAT 劫持：程序加载时，系统会将 IAT 中的函数地址替换为实际内存地址。汇编层面若直接修改 IAT 中的地址值，可实现函数调用重定向，导致程序执行非法操作。
3. 导入表伪造与混淆：通过构造虚假的导入表结构，例如伪造 IMAGE_THUNK_DATA 项，可绕过静态分析工具的检测，使程序在运行时动态加载恶意函数，或利用导入表解析逻辑漏洞触发内存破坏。

4. 延迟加载导入表 Delay-Load 漏洞：延迟加载机制下，导入表的解析发生在函数首次调用时，而非程序启动阶段。攻击者可利用延迟加载时的地址解析窗口，篡改未初始化的 IAT 项，实现运行时攻击。

8.2 典型攻击手段

1. 导入表覆盖攻击：通过汇编指令直接覆盖 IAT 中的函数地址，劫持 API 调用。例如将 user32.dll!MessageBoxA 的地址改为恶意 shellcode 的入口地址。
2. DLL 劫持的导入表利用：修改导入表中 DLL 名称的字符串指针，程序加载时会优先加载当前目录下的同名恶意 DLL，从而执行其中的导出函数。
3. 导入表解析逻辑漏洞利用：PE 文件解析导入表时，若对 IMAGE_IMPORT_DESCRIPTOR 的结束标志校验不严，攻击者可构造超长导入表项，导致汇编层面的内存越界访问，触发栈溢出或堆破坏。
4. 导入表混淆与反调试：通过汇编指令动态修改导入表的 FirstThunk 指针，干扰调试器对导入函数的解析，或使静态反汇编工具无法识别真实的导入函数。

8.3 汇编层面的防护机制

1. 导入表完整性校验：在汇编代码中加入导入表哈希校验逻辑，程序启动时验证导入表未被篡改，若异常则终止运行。
2. 导入表扁平化与内联：将关键 API 调用直接内联汇编实现，减少导入表暴露的函数依赖，降低劫持风险。
3. SEH 防护：在导入表解析的汇编代码段添加 SEH 异常处理，捕获因导入表篡改导致的内存访问异常，防止程序崩溃或被利用。
4. 利用系统防护机制：启用 Windows 的 SafeSEH、DEP（数据执行保护）或 CFG（控制流防护），限制对 IAT 的写入操作，或拦截非法的函数地址跳转。